

Fortran 演習: 提出課題 5

💡 ヒント

本課題は **AI エージェントの使用を前提** としている。

この課題では `main.f90` がテストプログラムである。テストとは、実装が仕様通りに動くかを確認するためのプログラムのことだ。まずは `build_and_test.sh` を実行し、うまく通らない箇所を直して、再び実行して確かめよう。実装と確認を少しずつ繰り返すやり方をテスト駆動開発 (TDD) という。

科学技術計算において、物理量を単位付きで扱うことは重要である。例えば「長さ」と「質量」を足すことはできないし、「速度」と「力」を比較することもできない。本課題では、物理量を単位付きで扱うためのモジュール `units` を実装しよう。

(0) 準備

プロジェクト構成

template/report5/ ディレクトリにあるファイルを自分の作業ディレクトリにコピーせよ：

```
report5/  
├── units.f90          # モジュールの実装  
├── main.f90          # テストプログラム  
└── build_and_test.sh # ビルド&テスト実行
```

ビルドとテスト

```
$ bash ./build_and_test.sh
```

(1) 基本課題

下記の仕様を満たす `units.f90` の実装を完成させよ。ただし `main.f90` は修正せずにそのまま使うこと。

仕様

派生型 `quantity_t`

物理量を表す派生型 `quantity_t` を定義せよ。

- **値:** 倍精度実数 `real(dp)`
- **次元:** 基本単位 (m, kg, s) の指数を表す長さ 3 の整数配列

例：速度 (m/s) は [1, 0, -1]、力 ($N = kg \cdot m/s^2$) は [1, 1, -2]

初期化関数 `quantity(val, unit_str)`

例えば以下のように使用する。

```
type(quantity_t) :: a, b, c  
a = quantity(10.0_dp, "m")      ! 10 メートル
```

```
b = quantity(5.0_dp, "kg")      ! 5 キログラム
c = quantity(3.0_dp, "m/s")     ! 3 メートル毎秒
```

単位文字列としてサポートする形式：

- 基本単位: "m", "kg", "s"
- 組み合わせ: "m/s", "m^2", "kg*m/s^2"
- 無次元: "" または "1"

算術演算子

演算子	説明
+, -	同じ次元同士の加減算。異なる次元が与えられた場合は NaN を値として返すこと
*	物理量同士，または実数との乗算
/	物理量同士，または実数との除算

NaN かどうかは `is_nan(q)` で判定できる。

例えば，異なる次元同士の加減算を試して `is_nan(q)` が `.true.` になることを確認するとよい。

(2) 発展課題（任意）

余力があれば追加で以下を実装してみよう。

- 単位文字列として SI 接頭辞 `u`（マイクロ），`m`（ミリ），`k`（キロ），`M`（メガ）を使えるようにし，`um`，`mm`，`km`，`Mg`，`ms` などを扱えるようにすること。
- `km/ms` や `Mg/m^3` のような，接頭辞つき単位を含む複合単位文字列も正しくパースできること。
- 加減算時には単位を自動で正規化し，`1000 mm + 1 m = 2 m` や `1000 ms + 1 s = 2 s` のような計算を可能にすること。
- `main.f90` に接頭辞を使うテストを追加し，そのテストが通るようにすること。
- 内部表現は自由とするが，`g` や `s` のように係数が異なるだけの量を正しく扱えること。